

Functional Documentation

MediQueue

1. Project Overview

MediQueue is a digital queue management platform designed for healthcare facilities.

The platform's main purpose is to help healthcare facilities manage patients waiting for medical services and automatically keep them informed of changes to their queue position through SMS notifications.

The platform is primarily intended for four types of users:

- Administrator
- Receptionist
- Doctor
- Patient

The patient does not need to create an account or install a mobile application in the MVP version.

2. Problem Statement

In some healthcare facilities, patients have to wait physically without knowing precisely:

- their position in the queue;
- how many people are ahead of them;
- how the waiting time is changing;
- when they need to go to the doctor's office.

Receptionists also have to manage patients manually, and doctors need a reliable way to know the order in which patients should be seen.

MediQueue centralizes queue management and automates patient notifications.

3. Objectives

Main Objective

Digitize medical queue management and automatically inform patients about changes in their position in the queue.

Secondary Objectives

- Centralize patient information.
- Centralize medical services.
- Assign doctors to their respective services.
- Automatically generate queue tickets.
- Manage patient order of service.
- Send SMS notifications to patients.
- Automatically notify the next patient.
- Allow doctors to manage their queues.
- Keep a history of consultations.
- Monitor the application and its infrastructure.

4. Actors

4.1 Administrator

Responsibilities:

- Manage medical services.
- Create doctor accounts.
- Create receptionist accounts.
- Activate or deactivate services.
- View statistics.
- View queues.

4.2 Receptionist

Responsibilities:

- Register patients.
- Select the requested service.
- Add patients to a queue.
- View the queue of a service.
- Search for patients or tickets.
- View the status of a patient.

4.3 Doctor

Responsibilities:

- View patients assigned to their service.
- Call the next patient.
- Start a consultation.
- Complete a consultation.
- Mark a patient as absent.
- View their patient history.

4.4 Patient

The patient does not need an account in the MVP.

The patient provides:

- First name.
- Last name.
- Phone number.
- Requested service.

The patient receives important information through SMS notifications.

5. General Workflow

Step 1 — Service Creation

The administrator creates the available medical services.

Examples:

- General Consultation
- Pediatrics
- Dentistry
- Cardiology
- Laboratory
- Radiology

A service can be:

- Active.
- Inactive.

Step 2 — Patient Registration

The receptionist registers the patient.

Required information:

- First name.
- Last name.
- Phone number.
- Requested service.

The system automatically creates a queue ticket.

Example:

Ticket: PED-024

The patient is added to the Pediatrics queue.

Step 3 — Initial Notification

The system automatically calculates the number of people ahead of the patient.

Example:

Ticket: PED-024

People ahead: 6

Position: 7

The patient receives an SMS:

MediQueue

You have been registered for the Pediatrics service.

There are currently 6 people ahead of you.

Your ticket: PED-024.

6. Queue Management

Example:

PED-021 Alice

PED-022 Marc

PED-023 Paul

PED-024 Jean

PED-025 Sarah

Jean has:

Position: 4

People ahead: 3

When the doctor calls Alice:

Alice → CALLED

Marc → 1 person ahead

Paul → 1 person ahead

Jean → 2 people ahead

Sarah → 3 people ahead

The system automatically updates the positions.

7. Calling a Patient

The doctor views their queue.

Example:

Pediatrics

1. Alice

2. Marc
3. Paul
4. Jean
5. Sarah

The doctor clicks:

CALL ALICE

Alice's ticket status becomes:

CALLED

Alice receives an SMS:

MediQueue

It is now your turn.

Please proceed to the Pediatrics service.

The other patients automatically receive updated information about their queue position.

8. Consultation

When the patient arrives:

CALLED

↓

IN_CONSULTATION

When the doctor finishes the consultation:

IN_CONSULTATION

↓

COMPLETED

The system can then allow the doctor to call the next patient.

9. Patient No-Show

If the patient does not show up:

CALLED

↓

NO_SHOW

The patient is removed from the active queue.

The system updates the remaining patients' positions.

10. Cancellation

A waiting ticket can be cancelled:

WAITING

↓

CANCELLED

The queue is recalculated automatically.

11. Ticket Statuses

The main ticket statuses are:

WAITING

CALLED

IN_CONSULTATION

COMPLETED

CANCELLED

NO_SHOW

Normal lifecycle:

WAITING

↓

CALLED

↓

IN_CONSULTATION

↓

COMPLETED

Alternative flows:

WAITING → CANCELLED

CALLED → NO_SHOW

12. SMS Notifications

The following events may trigger an SMS notification.

Patient Registration

Your ticket is PED-024.

There are 6 people ahead of you.

Position Changed

MediQueue Update:

There are now 3 people ahead of you.

Next Patient

MediQueue:

You are now the next patient.

Please remain available.

Patient Called

MediQueue:

It is now your turn.

Please proceed to the Pediatrics service.

13. MVP Features

Administration

- Authentication.
- Medical service management.
- Doctor management.
- Receptionist management.
- Service activation/deactivation.

Reception

- Register a patient.
- Select a service.
- Create a queue ticket.
- View the queue.
- Search for a patient or ticket.

Doctor

- View their queue.

- Call the next patient.
- Start a consultation.
- Complete a consultation.
- Mark a patient as a no-show.
- View consultation history.

Patient

- Receive SMS notifications.
- Know their ticket number.
- Know the number of people ahead of them.
- Receive queue position updates.
- Receive the notification when it is their turn.

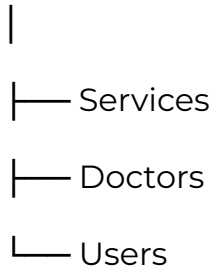
14. Features Outside the MVP

The following features may be added after the first version:

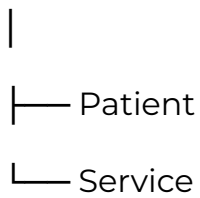
- Mobile application.
- WhatsApp notifications.
- Push notifications.
- Advanced waiting-time estimation.
- Advanced analytics and statistics.
- Multi-facility support.
- Appointment management.
- QR Code support.
- Public queue display screen.
- Payment integration.
- Medical priority management.
- Integration with electronic medical records.

15. Functional Architecture

ADMIN



RECEPTION



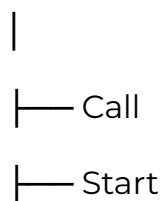
QUEUE TICKET

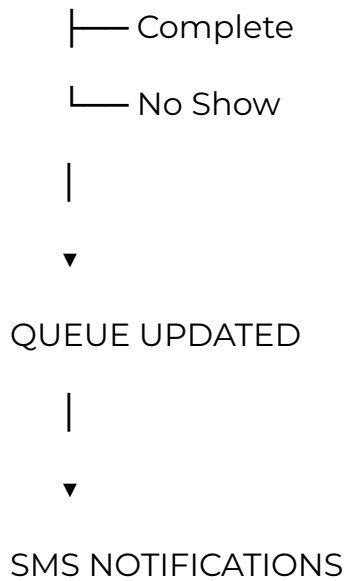


SMS NOTIFICATION



DOCTOR





16. Business Rules

Rule 1

A ticket can only be created for an active service.

Rule 2

A patient must provide a valid phone number to receive SMS notifications.

Rule 3

A doctor can only manage patients belonging to their assigned service.

Rule 4

Under normal operation, only the first patient in the waiting queue can be called.

Rule 5

A **COMPLETED** ticket cannot be called again.

Rule 6

A **CANCELLED** ticket cannot be called again.

Rule 7

A **NO_SHOW** ticket cannot be called again.

Rule 8

When a patient leaves the active queue, the positions of the remaining patients are recalculated.

Rule 9

When a relevant queue position changes, the system generates a notification.

Rule 10

SMS delivery must be asynchronous so that the API request is not blocked while messages are being sent.

17. Domain Events

The application will use domain events to decouple queue management from notification processing.

Examples include:

PatientRegistered

PatientAddedToQueue

PatientCalled

PatientEnteredConsultation

PatientConsultationCompleted

PatientCancelled

PatientMarkedAsNoShow

QueuePositionChanged

Example:

PatientCalled

|

▼

QueuePositionChanged

|

└─▶ Notify next patient

└─▶ Notify patients whose positions changed

└─▶ Update notification jobs

18. SMS Processing

SMS sending should not directly block the main API request.

The application should use an asynchronous job/queue mechanism.

Example:

Doctor calls patient

|

▼

Laravel API

|

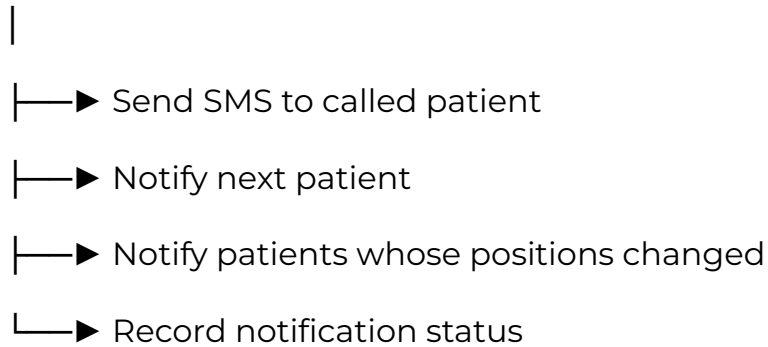
▼

PatientCalled event

|



Queue Job



This architecture allows the application to remain responsive even when multiple SMS messages need to be sent.

19. SMS Provider Abstraction

The application should not directly depend on a single SMS provider.

An abstraction should be created:

SmsServiceInterface

Possible implementations:

SmsServiceInterface



During development, **FakeSmsService** can be used to simulate SMS delivery.

Example:

SMS SENT

To: +237XXXXXXXXXX

Message:

You are now the next patient.

Please remain available.

In production, the application can be connected to an appropriate SMS provider.

20. Data Model

Users

users

id

name

email

password

role

service_id

created_at

updated_at

Patients

patients

id

first_name

last_name

phone

created_at

updated_at

Services

services

id

name

description

is_active

created_at

updated_at

Queue Tickets

queue_tickets

id

ticket_number

patient_id

service_id

status

position

created_at

called_at

started_at

completed_at

Notifications

notifications

id

patient_id

ticket_id

type

phone

message

status

sent_at

created_at

Future tables may include:

notification_logs

consultations

audit_logs

21. DDD Architecture

The main bounded contexts are:

MediQueue

|

├─ Identity

|

├─ Patient

|

├─ Service

|

├─ Queue

|

├─ Consultation

|

└─ Notification

The core business domain is:

Queue Management

The main domain relationship is:

Patient

|

▼

QueueTicket

|

├— Service

├— Position

└— Status

Domain events connect queue operations to notifications:

QueueTicket

|

├— PatientCalled

| └— Notification

|

├— QueuePositionChanged

| └— Notification

|

└— ConsultationCompleted

 └— NextPatient

22. API Requirements

Administration

POST /api/v1/services

GET /api/v1/services

PUT /api/v1/services/{id}

DELETE /api/v1/services/{id}

Reception

POST /api/v1/patients

POST /api/v1/queue/tickets

GET /api/v1/queues/{serviceId}

Doctor

GET /api/v1/doctor/queue

POST /api/v1/tickets/{id}/call

POST /api/v1/tickets/{id}/start

POST /api/v1/tickets/{id}/complete

POST /api/v1/tickets/{id}/no-show

Patient

GET /api/v1/tickets/{id}/status

23. Frontend Requirements

The frontend will contain three main interfaces.

Administration

/admin

Dashboard

Services

Doctors

Users

Statistics

Reception

/reception

Register Patient

First Name

Last Name

Phone Number

Service

[REGISTER PATIENT]

Doctor

/doctor

My Service: Pediatrics

Current Patient

Alice

Waiting Queue

#12 Marc

#13 Paul

#14 Jean

#15 Sarah

[CALL NEXT PATIENT]

24. Technology Stack

The planned technology stack is:

Frontend

- React
- TypeScript

Backend

- Laravel
- PHP
- REST API

Database

- PostgreSQL

Architecture

- Domain-Driven Design
- Clean Architecture principles
- Domain Events

Infrastructure

- Docker
- Docker Compose
- Nginx

CI/CD

- Git
- GitHub
- GitHub Actions

Cloud

- AWS EC2

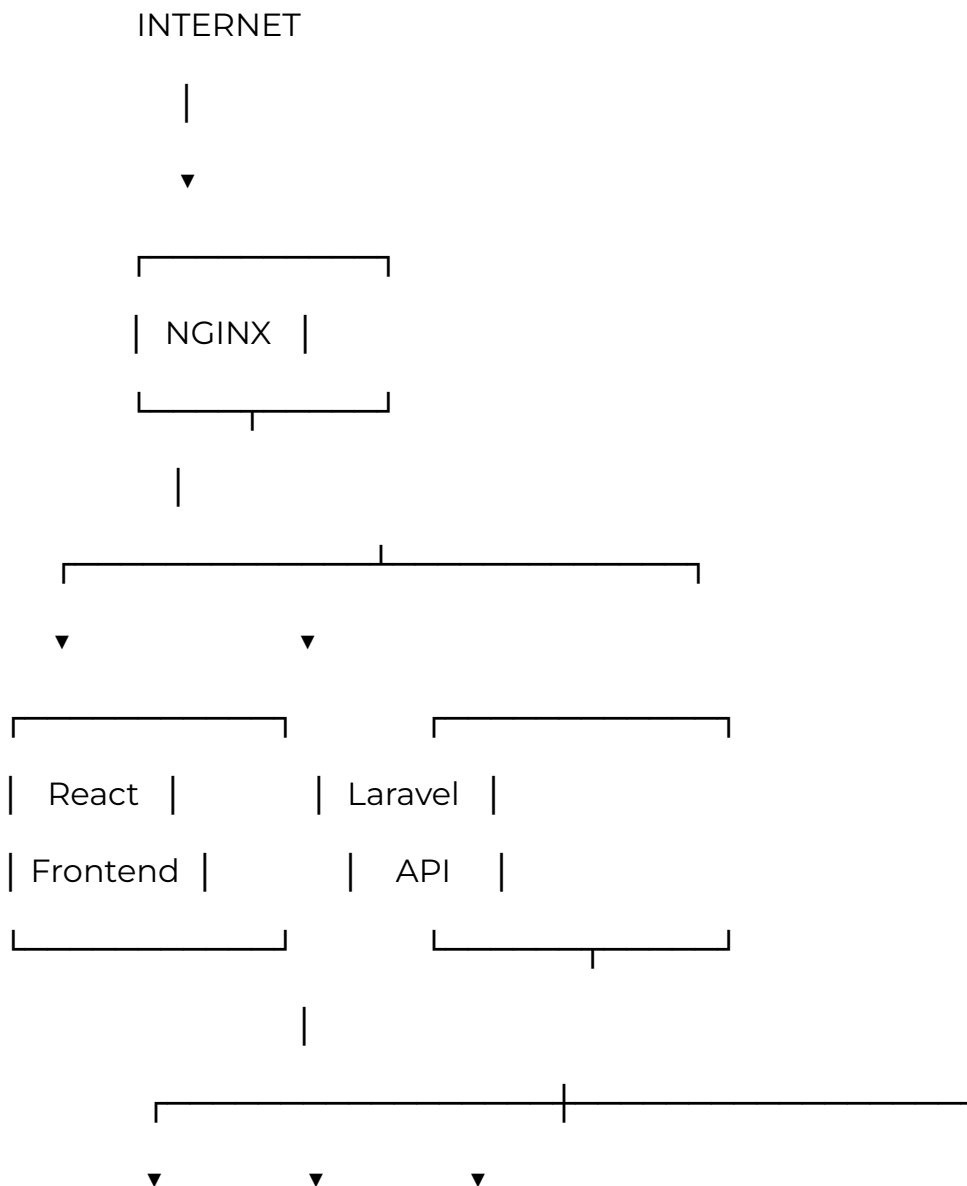
Security

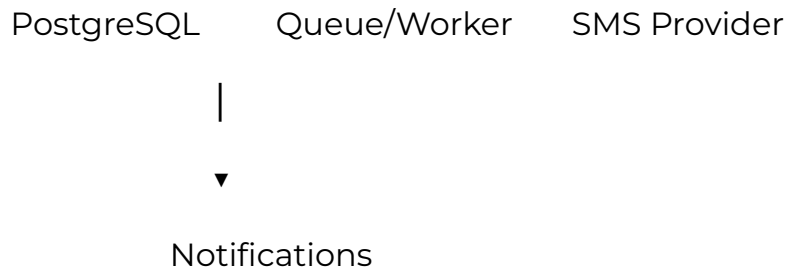
- HTTPS
- Let's Encrypt
- Authentication
- Role-Based Access Control

Monitoring

- Prometheus
- Grafana

25. Production Architecture





The production environment will run on AWS EC2 using Docker containers.

Monitoring will be provided by Prometheus and Grafana.

26. MVP Success Criteria

The MVP will be considered functional when the following scenario works end-to-end:

1. The administrator creates the Pediatrics service.
2. The administrator creates a doctor account.
3. The receptionist registers a patient.
4. The receptionist selects Pediatrics.
5. The system creates a queue ticket.
6. The patient receives the initial SMS.
7. The doctor sees the patient in their queue.
8. The doctor calls the patient.
9. The patient receives the call notification.
10. The positions of the other patients are recalculated.
11. The relevant patients receive queue position updates.
12. The doctor starts the consultation.
13. The doctor completes the consultation.
14. The system allows the next patient to be called.
15. All relevant operations are logged.
16. The application is deployed using Docker on AWS EC2.
17. The application is accessible through HTTPS.
18. Application and infrastructure metrics are available through Grafana.

27. Future Improvements

Potential future versions may introduce:

- Mobile applications.
- WhatsApp integration.
- Push notifications.
- Real-time WebSocket updates.
- AI-based waiting-time estimation.
- Multi-hospital support.
- Appointment scheduling.
- QR code check-in.
- Public queue display.
- Advanced analytics.
- SMS delivery analytics.
- Patient priority management.
- Electronic medical record integration.
- High availability infrastructure.
- AWS managed database services.
- Auto-scaling.
- Infrastructure as Code with Terraform.